4. It changes some details for the new submission and upload a new package for the submission to Azure Blob Storage.
5. Next, it updates and then commits the new submission to Partner Center.
6. Finally, it periodically checks the status of the new submission until the submission is successfully committed.

Python

```
import http.client, json, requests, time
from azure.storage.blob import BlobClient
accessToken = ""  # Your access token
applicationId = ""  # Your application ID
appSubmissionRequestJson = "";  # Your submission request JSON
zipFilePath = r'*.zip'  # Your zip file path

headers = {"Authorization": "Bearer " + accessToken,
           "Content-type": "application/json",
           "User-Agent": "Python"}
ingestionConnection =
http.client.HTTPSConnection("manage.devcenter.microsoft.com")

# Get application
ingestionConnection.request("GET",
"/v1.0/my/applications/{0}".format(applicationId), "", headers)
appResponse = ingestionConnection.getresponse()
print(appResponse.status)
print(appResponse.headers["MS-CorrelationId"])  # Log correlation ID

# Delete existing in-progress submission
appJsonObject = json.loads(appResponse.read().decode())
if "pendingApplicationSubmission" in appJsonObject :
    submissionToRemove = appJsonObject["pendingApplicationSubmission"]["id"]
    ingestionConnection.request("DELETE",
"/v1.0/my/applications/{0}/submissions/{1}".format(applicationId,
submissionToRemove), "", headers)
    deleteSubmissionResponse = ingestionConnection.getresponse()
    print(deleteSubmissionResponse.status)
    print(deleteSubmissionResponse.headers["MS-CorrelationId"])  # Log
correlation ID
    deleteSubmissionResponse.read()

# Create submission
ingestionConnection.request("POST",
"/v1.0/my/applications/{0}/submissions".format(applicationId), "", headers)
createSubmissionResponse = ingestionConnection.getresponse()
print(createSubmissionResponse.status)
print(createSubmissionResponse.headers["MS-CorrelationId"])  # Log
correlation ID

submissionJsonObject = json.loads(createSubmissionResponse.read().decode())
submissionId = submissionJsonObject["id"]
fileUploadUrl = submissionJsonObject["fileUploadUrl"]
print(submissionId)
```

```
print(fileUploadUrl)

# Update submission
ingestionConnection.request("PUT",
"/v1.0/my/applications/{0}/submissions/{1}".format(applicationId,
submissionId), appSubmissionRequestJson, headers)
updateSubmissionResponse = ingestionConnection.getresponse()
print(updateSubmissionResponse.status)
print(updateSubmissionResponse.headers["MS-CorrelationId"])  # Log
correlation ID
updateSubmissionResponse.read()

# Upload images and packages in a zip file.
blob_client = BlobClient.from_blob_url(fileUploadUrl)
with open(zipFilePath, "rb") as data:
    blob_client.upload_blob(data, blob_type="BlockBlob")

# Commit submission
ingestionConnection.request("POST",
"/v1.0/my/applications/{0}/submissions/{1}/commit".format(applicationId,
submissionId), "", headers)
commitResponse = ingestionConnection.getresponse()
print(commitResponse.status)
print(commitResponse.headers["MS-CorrelationId"])  # Log correlation ID
print(commitResponse.read())

# Pull submission status until commit process is completed
ingestionConnection.request("GET",
"/v1.0/my/applications/{0}/submissions/{1}/status".format(applicationId,
submissionId), "", headers)
getSubmissionStatusResponse = ingestionConnection.getresponse()
submissionJsonObject =
json.loads(getSubmissionStatusResponse.read().decode())
while submissionJsonObject["status"] == "CommitStarted":
    time.sleep(60)
    ingestionConnection.request("GET",
"/v1.0/my/applications/{0}/submissions/{1}/status".format(applicationId,
submissionId), "", headers)
    getSubmissionStatusResponse = ingestionConnection.getresponse()
    submissionJsonObject =
json.loads(getSubmissionStatusResponse.read().decode())
    print(submissionJsonObject["status"])

print(submissionJsonObject["status"])
print(submissionJsonObject)

ingestionConnection.close()
```

# Create an add-on submission

The following example shows how to use several methods in the Microsoft Store submission API to create an add-on submission. To do this, the code creates a new

submission as a clone of the last published submission, and then updates and commits the cloned submission to Partner Center. Specifically, the example performs these tasks:

1. To begin, the example gets data for the specified add-on.
2. Next, it deletes the pending submission for the add-on, if one exists.
3. It then creates a new submission for the add-on (the new submission is a copy of the last published submission).
4. It uploads a ZIP archive that contains icons for the submission to Azure Blob Storage. For more information, see the relevant instructions about uploading a ZIP archive to Azure Blob Storage in Create an add-on submission.
5. Next, it updates and then commits the new submission to Partner Center.
6. Finally, it periodically checks the status of the new submission until the submission is successfully committed.

Python

```
import http.client, json, requests, time
from azure.storage.blob import BlobClient
accessToken = ""  # Your access token
inAppProductId = ""  # Your in-app-product ID
updateSubmissionRequestBody = "";  # Your in-app-product submission request
JSON
zipFilePath = r'*.zip'  # Your zip file path

headers = {"Authorization": "Bearer " + accessToken,
           "Content-type": "application/json",
           "User-Agent": "Python"}
ingestionConnection =
http.client.HTTPSConnection("manage.devcenter.microsoft.com")

# Get in-app-product
ingestionConnection.request("GET",
"/v1.0/my/inappproducts/{0}".format(inAppProductId), "", headers)
iapResponse = ingestionConnection.getresponse()
print(iapResponse.status)
print(iapResponse.headers["MS-CorrelationId"])  # Log correlation ID

# Delete existing in-progress submission
iapJsonObject = json.loads(iapResponse.read().decode())
if "pendingInAppProductSubmission" in iapJsonObject :
    submissionToRemove = iapJsonObject["pendingInAppProductSubmission"]
["id"]
    ingestionConnection.request("DELETE",
"/v1.0/my/inappproducts/{0}/submissions/{1}".format(inAppProductId,
submissionToRemove), "", headers)
    deleteSubmissionResponse = ingestionConnection.getresponse()
    print(deleteSubmissionResponse.status)
    print(deleteSubmissionResponse.headers["MS-CorrelationId"])  # Log
correlation ID
    deleteSubmissionResponse.read()
```

```
# Create submission
ingestionConnection.request("POST",
"/v1.0/my/inappproducts/{0}/submissions".format(inAppProductId), "",
headers)
createSubmissionResponse = ingestionConnection.getresponse()
print(createSubmissionResponse.status)
print(createSubmissionResponse.headers["MS-CorrelationId"])  # Log
correlation ID

submissionJsonObject = json.loads(createSubmissionResponse.read().decode())
submissionId = submissionJsonObject["id"]
fileUploadUrl = submissionJsonObject["fileUploadUrl"]
print(submissionId)
print(fileUploadUrl)

# Update submission
ingestionConnection.request("PUT",
"/v1.0/my/inappproducts/{0}/submissions/{1}".format(inAppProductId,
submissionId), updateSubmissionRequestBody, headers)
updateSubmissionResponse = ingestionConnection.getresponse()
print(updateSubmissionResponse.status)
print(updateSubmissionResponse.headers["MS-CorrelationId"])  # Log
correlation ID
updateSubmissionResponse.read()

# Upload images and packages in a zip file.
blob_client = BlobClient.from_blob_url(fileUploadUrl)
with open(zipFilePath, "rb") as data:
    blob_client.upload_blob(data, blob_type="BlockBlob")

# Commit submission
ingestionConnection.request("POST",
"/v1.0/my/inappproducts/{0}/submissions/{1}/commit".format(inAppProductId,
submissionId), "", headers)
commitResponse = ingestionConnection.getresponse()
print(commitResponse.status)
print(commitResponse.headers["MS-CorrelationId"])  # Log correlation ID
print(commitResponse.read())

# Pull submission status until commit process is completed
ingestionConnection.request("GET",
"/v1.0/my/inappproducts/{0}/submissions/{1}/status".format(inAppProductId,
submissionId), "", headers)
getSubmissionStatusResponse = ingestionConnection.getresponse()
submissionJsonObject =
json.loads(getSubmissionStatusResponse.read().decode())
while submissionJsonObject["status"] == "CommitStarted":
    time.sleep(60)
    ingestionConnection.request("GET",
"/v1.0/my/inappproducts/{0}/submissions/{1}/status".format(inAppProductId,
submissionId), "", headers)
    getSubmissionStatusResponse = ingestionConnection.getresponse()
    submissionJsonObject =
json.loads(getSubmissionStatusResponse.read().decode())
```

```
    print(submissionJsonObject["status"])

print(submissionJsonObject["status"])
print(submissionJsonObject)

ingestionConnection.close()
```

# Create a package flight submission

The following example shows how to use several methods in the Microsoft Store submission API to create a package flight submission. To do this, the code creates a new submission as a clone of the last published submission, and then updates and commits the cloned submission to Partner Center. Specifically, the example performs these tasks:

1. To begin, the example gets data for the specified package flight.
2. Next, it deletes the pending submission for the package flight, if one exists.
3. It then creates a new submission for the package flight (the new submission is a copy of the last published submission).
4. It uploads a new package for the submission to Azure Blob Storage. For more information, see the relevant instructions about uploading a ZIP archive to Azure Blob Storage in Create a package flight submission.
5. Next, it updates and then commits the new submission to Partner Center.
6. Finally, it periodically checks the status of the new submission until the submission is successfully committed.

Python

```
import http.client, json, requests, time, zipfile
from azure.storage.blob import BlobClient
accessToken = ""  # Your access token
applicationId = ""  # Your application ID
flightId = ""  # Your flight ID
flightSubmissionRequestJson = ""  # Your submission request JSON
zipFilePath = r'*.zip'  # Your zip file path

headers = {"Authorization": "Bearer " + accessToken,
           "Content-type": "application/json",
           "User-Agent": "Python"}
ingestionConnection =
http.client.HTTPSConnection("manage.devcenter.microsoft.com")

# Get flight
ingestionConnection.request("GET",
"/v1.0/my/applications/{0}/flights/{1}".format(applicationId, flightId), "",
headers)
flightResponse = ingestionConnection.getresponse()
print(flightResponse.status)
```

```
print(flightResponse.headers["MS-CorrelationId"])  # Log correlation ID

# Delete existing in-progress submission
flightJsonObject = json.loads(flightResponse.read().decode())
if "pendingFlightSubmission" in flightJsonObject :
    submissionToRemove = flightJsonObject["pendingFlightSubmission"]["id"]
    ingestionConnection.request("DELETE",
"/v1.0/my/applications/{0}/flights/{1}/submissions/{2}".format(applicationId
, flightId, submissionToRemove), "", headers)
    deleteSubmissionResponse = ingestionConnection.getresponse()
    print(deleteSubmissionResponse.status)
    print(deleteSubmissionResponse.headers["MS-CorrelationId"])  # Log
correlation ID
    deleteSubmissionResponse.read()

# Create submission
ingestionConnection.request("POST",
"/v1.0/my/applications/{0}/flights/{1}/submissions".format(applicationId,
flightId), "", headers)
createSubmissionResponse = ingestionConnection.getresponse()
print(createSubmissionResponse.status)
print(createSubmissionResponse.headers["MS-CorrelationId"])  # Log
correlation ID

submissionJsonObject = json.loads(createSubmissionResponse.read().decode())
submissionId = submissionJsonObject["id"]
fileUploadUrl = submissionJsonObject["fileUploadUrl"]
print(submissionId)
print(fileUploadUrl)

# Update submission
ingestionConnection.request("PUT",
"/v1.0/my/applications/{0}/flights/{1}/submissions/{2}".format(applicationId
, flightId, submissionId), flightSubmissionRequestJson, headers)
updateSubmissionResponse = ingestionConnection.getresponse()
print(updateSubmissionResponse.status)
print(updateSubmissionResponse.headers["MS-CorrelationId"])  # Log
correlation ID
updateSubmissionResponse.read()

# Upload images and packages in a zip file.
blob_client = BlobClient.from_blob_url(fileUploadUrl)
with open(zipFilePath, "rb") as data:
    blob_client.upload_blob(data, blob_type="BlockBlob")

# Commit submission
ingestionConnection.request("POST",
"/v1.0/my/applications/{0}/flights/{1}/submissions/{2}/commit".format(applic
ationId, flightId, submissionId), "", headers)
commitResponse = ingestionConnection.getresponse()
print(commitResponse.status)
print(commitResponse.headers["MS-CorrelationId"])  # Log correlation ID
print(commitResponse.read())

# Pull submission status until commit process is completed
```

```
ingestionConnection.request("GET",
"/v1.0/my/applications/{0}/flights/{1}/submissions/{2}/status".format(applic
ationId, flightId, submissionId), "", headers)
getSubmissionStatusResponse = ingestionConnection.getresponse()
submissionJsonObject =
json.loads(getSubmissionStatusResponse.read().decode())
while submissionJsonObject["status"] == "CommitStarted":
    time.sleep(60)
    ingestionConnection.request("GET",
"/v1.0/my/applications/{0}/flights/{1}/submissions/{2}/status".format(applic
ationId, flightId, submissionId), "", headers)
    getSubmissionStatusResponse = ingestionConnection.getresponse()
    submissionJsonObject =
json.loads(getSubmissionStatusResponse.read().decode())
    print(submissionJsonObject["status"])

print(submissionJsonObject["status"])
print(submissionJsonObject)

ingestionConnection.close()
```

# Related topics

- Create and manage submissions using Microsoft Store services

# Python sample: app submission with game options and trailers

Article • 01/04/2023

This article provides Python code examples that demonstrate how to use the Microsoft Store submission API for these tasks:

- Obtain an Azure AD access token to use with the Microsoft Store submission API.
- Create an app submission
- Configure Store listing data for the app submission, including the gaming and trailers advanced listing options.
- Upload the ZIP file containing the packages, listing images, and trailer files for the app submission.
- Commit the app submission.

## Create an app submission

This code calls other example classes and functions to use the Microsoft Store submission API to create and commit an app submission that contains game options and a trailer. To adapt this code for your own use:

- Assign the `tenant` variable to the tenant ID for your app, and assign the `client` and `secret` variables to the client ID and key for your app. For more information, see How to associate an Azure AD application with your Partner Center account
- Assign the `application_id` variable to the Store ID of the app for which you want to create a submission.

Python

```
import time

from devcenterclient import DevCenterClient, DevCenterAccessTokenClient
import submissiondatasamples as samples

# Add your tenant ID, client ID, and client secret here.
tenant = ""
client = ""
secret = ""
acc_token_client = DevCenterAccessTokenClient(tenant, client, secret)

acc_token =
acc_token_client.get_access_token("https://manage.devcenter.microsoft.com")
dev_center = DevCenterClient("manage.devcenter.microsoft.com", acc_token)
```

```
# The application ID is taken from your app dashboard page's URI in Dev
Center,
# e.g. https://developer.microsoft.com/en-
us/dashboard/apps/{application_id}/
application_id = ""

# Get the application object, and cancel any in progress submissions.
is_ok, app = dev_center.get_application(application_id)
assert is_ok

if "pendingApplicationSubmission" in app:
    in_progress_submission_id = app["pendingApplicationSubmission"]["id"]
    is_ok = dev_center.cancel_in_progress_submission(application_id,
in_progress_submission_id)
    assert is_ok

# Create a new submission, based on the last published submission.
is_ok, submission = dev_center.create_submission(application_id)
assert is_ok
submission_id = submission["id"]

# The following fields are required:
submission["applicationCategory"] = "Games_Fighting"
submission["listings"] = samples.get_listings_object()
submission["Pricing"] = samples.get_pricing_object()
submission["packages"] = [samples.get_package_object()]
submission["allowTargetFutureDeviceFamilies"] =
samples.get_device_families_object()

# The app must have the hasAdvancedListingPermission set to True in order
for gaming options
# and trailers to be applied. If that's not the case, you can still update
the app and
# its submissions through the API, but gaming options and trailers won't be
saved.
if not "hasAdvancedListingPermission" in app or not
app["hasAdvancedListingPermission"]:
    print("This application does not support gaming options or trailers.")
else:
    submission["gamingOptions"] = [samples.get_gaming_options_object()]
    submission["trailers"] = [samples.get_trailer_object()]

# Continue updating the submission_json object with additional options as
needed.
# After you've finished, call the Update API with the code below to save it:
is_ok, submission = dev_center.update_submission(application_id,
submission_id, submission)
assert is_ok

# All images and packages should be located in a single ZIP file. In the
submission JSON,
# the file names for all objects requiring them (icons, packages, etc.) must
exactly
# match the file names from the ZIP file.
```

```
zip_file_path = ""
is_ok = dev_center.upload_zip_file_for_submission(application_id,
submission_id, zip_file_path)
assert is_ok

# Committing the submission will start the submission process for it. Once
committed,
# the submission can no longer be changed.
is_ok = dev_center.commit_submission(application_id, submission_id)
assert is_ok

# After committing, you can poll the commit API for the status of the
submission's process using
# the following code.
waiting_for_commit_start = True
while waiting_for_commit_start:
    is_ok, submission_status =
dev_center.get_submission_status(application_id, submission_id)
    assert is_ok
    waiting_for_commit_start = submission_status == "CommitStarted"
    if waiting_for_commit_start:
        time.sleep(60)
```

# Obtain an Azure AD access token and invoke the submission API

The following example defines the following classes:

- The `DevCenterAccessTokenClient` class defines a helper method that uses the your `tenantId`, `clientId` and `clientSecret` values to create an Azure AD access token to use with the Microsoft Store submission API.
- The `DevCenterClient` class defines helper methods that invoke a variety of methods in the Microsoft Store submission API and upload the ZIP file containing the packages, listing images, and trailer files for the app submission.

Python

```
import http.client
import json
import requests

class DevCenterAccessTokenClient(object):
    """A client for acquiring access tokens from AAD to use with the Dev
Center Client."""
    def __init__(self, tenant_id, client_id, client_secret):
        self.tenant_id = tenant_id
        self.client_id = client_id
        self.client_secret = client_secret
```

```
    def get_access_token(self, resource):
        """Acquires an access token to the specific resource via the AAD
tenant."""
        body_format = "grant_type=client_credentials&client_id=
{0}&client_secret={1}&resource={2}"
        body = body_format.format(self.client_id, self.client_secret,
resource)
        access_headers = {"Content-Type": "application/x-www-form-
urlencoded; charset=utf-8"}
        token_conn =
http.client.HTTPSConnection("login.microsoftonline.com")
        token_relative_path = "/{0}/oauth2/token".format(self.tenant_id)
        token_conn.request("POST", token_relative_path, body,
headers=access_headers)

        token_response = token_conn.getresponse()
        token_json = json.loads(token_response.read().decode())
        token_conn.close()
        return token_json["access_token"]

class DevCenterClient(object):
    """A client for the Dev Center API."""
    def __init__(self, base_uri, access_token):
        self.base_uri = base_uri
        self.request_headers = {
            "Authorization": "Bearer " + access_token,
            "Content-type": "application/json",
            "User-Agent": "Python"
        }

    def get_application(self, application_id):
        """Returns the application as defined in Dev Center."""
        path = "/v1.0/my/applications/{0}".format(application_id)
        return self._get(path)

    def cancel_in_progress_submission(self, application_id, submission_id):
        """Cancels the in-progress submission."""
        path =
"/v1.0/my/applications/{0}/submissions/{1}".format(application_id,
submission_id)
        return self._delete(path)

    def create_submission(self, application_id):
        """Creates a new submission in Dev Center. This is identical to
clicking
        the Create Submission button in Dev Center."""
        path =
"/v1.0/my/applications/{0}/submissions".format(application_id)
        return self._post(path)

    def update_submission(self, application_id, submission_id, submission):
        """Updates the submission in Dev Center using the JSON provided."""
        path = "/v1.0/my/applications/{0}/submissions/{1}"
        path = path.format(application_id, submission_id)
```

```
        return self._put(path, submission)

    def get_submission(self, application_id, submission_id):
        """Gets the submission in Dev Center."""
        path = "/v1.0/my/applications/{0}/submissions/{1}"
        path = path.format(application_id, submission_id)
        return self._get(path)

    def commit_submission(self, application_id, submission_id):
        """Commits the submission to Dev Center. Once committed, Dev Center
will
        begin processing the submission and verify package integrity and
send
        it for certification."""
        path = "/v1.0/my/applications/{0}/submissions/{1}/commit"
        path = path.format(application_id, submission_id)
        return self._post(path)

    def get_submission_status(self, application_id, submission_id):
        """Returns the current state of the submission in Dev Center,
        such as is the submission in certification, committed, publishing,
        etc."""
        path = "/v1.0/my/applications/{0}/submissions/{1}/status"
        path = path.format(application_id, submission_id)
        response_ok, response_obj = self._get(path)
        if "status" in response_obj:
            return (response_ok, response_obj["status"])
        else:
            return (response_ok, "Unknown")

    def upload_zip_file_for_submission(self, application_id, submission_id,
zip_file_path):
        """Uploads a ZIP file for the Submission API for the submission
object."""
        is_ok, submission = self.get_submission(application_id,
submission_id)
        if not is_ok:
            raise "Failed to get submission."

        zip_file = open(zip_file_path, 'rb')
        upload_uri = submission["fileUploadUrl"].replace("+", "%2B")
        upload_headers = {"x-ms-blob-type": "BlockBlob"}
        upload_response = requests.put(upload_uri, zip_file,
headers=upload_headers)
        upload_response.raise_for_status()

    def _get(self, path):
        return self._invoke("GET", path)

    def _post(self, path, obj=None):
        return self._invoke("POST", path, obj)

    def _put(self, path, obj=None):
        return self._invoke("PUT", path, obj)
```

```
    def _delete(self, path):
        return self._invoke("DELETE", path)

    def _invoke(self, method, path, obj=None):
        body = ""
        if not obj is None:
            body = json.dumps(obj)
        conn = http.client.HTTPSConnection(self.base_uri)
        conn.request(method, path, body, self.request_headers)
        response = conn.getresponse()
        response_body = response.read().decode()
        response_body_length = int(response.headers["Content-Length"])
        response_obj = None
        if not response_body is None and response_body_length != 0:
            response_obj = json.loads(response_body)
        response_ok = self._response_ok(response)
        conn.close()
        return (response_ok, response_obj)

    def _response_ok(self, response):
        status_code = int(response.status)
        return status_code >= 200 and status_code <= 299
```

# Get app submission listing data

The following example defines helper functions that return JSON-formatted listing data for a new sample app submission.

Python

```
def get_listings_object():
    """Gets a sample listings map for a submission."""
    listings = {
        # Each listing is targeted at a specific language-locale code, e.g.
EN-US.
        "en-us" : {
            # This structure holds basic information to display in the
store.
            "baseListing" : {
                "copyrightAndTrademarkInfo" : "(C) 2017 Microsoft",
                # Up to 7 keywords may be provided in a listing.
                "keywords"  : ["SampleApp", "SampleFightingGame",
"GameOptions"],
                "licenseTerms" : "http://example.com/licenseTerms.aspx",
                "privacyPolicy" : "http://example.com/privacyPolicy.aspx",
                "supportContact" : "support@example.com",
                "websiteUrl" : "http://example.com",
                "description" : "A sample game showing off gameplay options
code.",
                "features" : ["Doesn't crash", "Likes to eat chips"],
                "releaseNotes" : "Initial release",
```

```
                "recommendedHardware" : [],
                # If your app works better with specific hardware (or needs
it), you can
                # add or update values here.
                "hardwarePreferences": ["Keyboard", "Mouse"],
                # The title of the app must match a reserved name for the
app in Dev Center.
                # If it doesn't, attempting to update the submission will
fail.
                "title" : "Super Dev Center API Simulator 2017",
                "images" : [
                    # There are several types of images available; at least
one screenshot
                    # is required.
                    {
                        # The file name is relative to the root of the
uploaded ZIP file.
                        "fileName" : "img/screenshot.png",
                        "description" : "A basic screenshot of the app.",
                        "imageType" : "Screenshot"
                    }
                ]
            },
            # If there are any specific overrides to above information for
Windows 8,
            # Windows 8.1, Windows Phone 7.1, 8.0, or 8.1, you can add
information here.
            "platformOverrides" : {}
        }
    }
    return listings

def get_package_object():
    """Gets a sample package for the submission in Dev Center."""
    package = {
        # The file name is relative to the root of the uploaded ZIP file.
        "fileName" : "bin/super_dev_ctr_api_sim.appxupload",
        # If you haven't begun to upload the file yet, set this value to
"PendingUpload".
        "fileStatus" : "PendingUpload"
    }
    return package

def get_pricing_object():
    """Gets a sample pricing object for a submission."""
    pricing = {
        # How long the trial period is, if one is allowed. Valid values are
NoFreeTrial,
        # OneDay, SevenDays, FifteenDays, ThirtyDays, or TrialNeverExpires.
        "trialPeriod" : "NoFreeTrial",
        # Maps to the default price for the app.
        "priceId" : "Free",
        # If you'd like to offer your app in different markets at different
prices, you
        # can provide priceId values per language/locale code.
```

```
        "marketSpecificPricing" : {}
    }
    return pricing

def get_device_families_object():
    """Gets a sample device families object for a submission."""
    device_families = {
        # Supported values are Desktop, Mobile, Xbox, and Holographic. To
make
        # the app available on that specific platform, set the value to
True.
        "Desktop" : True,
        "Mobile" : False,
        "Xbox" : True,
        "Holographic" : False
    }
    return device_families

def get_gaming_options_object():
    """Gets a sample gaming options object for a submission."""
    gaming_options = {
        # The genres of your app.
        "Genres" : ["Games_Fighting"],

        # Set this to True if your game supports local multiplayer. This
field is required.
        "IsLocalMultiplayer" : True,

        # If local multiplayer is supported, you must provide the minimum
and maximum players
        # supported. Valid values are between 2 and 1000 inclusive.
        "LocalMultiplayerMinPlayers" : 2,
        "LocalMultiplayerMaxPlayers" : 4,

        # Set this to True if your game supports local co-op play. This
field is required.
        "IsLocalCooperative" : True,

        # If local co-op is supported, you must provide the minimum and
maximum players
        # supported. Valid values are between 2 and 1000 inclusive.
        "LocalCooperativeMinPlayers" : 2,
        "LocalCooperativeMaxPlayers" : 4,

        # Set this to True if your game supports online multiplayer. This
field is required.
        "IsOnlineMultiplayer" : True,

        # If online multiplayer is supported, you must provide the minimum
and maximum players
        # supported. Valid values are between 2 and 1000 inclusive.
        "OnlineMultiplayerMinPlayers" : 2,
        "OnlineMultiplayerMaxPlayers" : 4,

        # Set this to true if your game supports online co-op play. This
```

```
field is required.
        "IsOnlineCooperative" : True,

        # If online co-op is supported, you must provide the minimum and
maximum players
        # supported. Valid values are between 2 and 1000 inclusive.
        "OnlineCooperativeMinPlayers" : 2,
        "OnlineCooperativeMaxPlayers" : 4,

        # If your game supports broadcasting a stream to other players, set
this field to True.
        # The field is required.
        "IsBroadcastingPrivilegeGranted" : True,

        # If your game supports cross-device play (e.g. a player can play on
an Xbox One with
        # their friend who's playing on a PC), set this field to True. This
field is required.
        "IsCrossPlayEnabled" : True,

        # If your game supports Kinect usage, set this field to "Enabled",
otherwise, set it to
        # "Disabled". This field is required.
        "KinectDataForExternal" : "Disabled",

        # Free text about any other peripherals that your game supports.
This field is optional.
        "OtherPeripherals" : "Supports the usage of all fighting joysticks."
    }
    return gaming_options

def get_trailer_object():
    """Gets a sample trailer object for the submission in Dev Center."""
    trailer = {
        # This is the filename of the trailer. The file name is a relative
path to the
        # root of the ZIP file to be uploaded to the API.
        "VideoFileName" : "trailers/main/my_awesome_trailer.mpeg",

        # Aside from the video itself, a trailer can have image assets such
as screenshots
        # or alternate images. These are separated by language-locale code,
e.g. EN-US.
        "TrailerAssets" : {
            "en-us" : {

                # The title of the trailer to display in the store.
                "Title" : "Main Trailer",

                # The list of images provided with the trailer that are
shown
                # when the trailer isn't playing.
                "ImageList" : [
                    {
                        # The file name of the image. The file name is a
```

```
relative
                        # path to the root of the ZIP
                        # file to be uploaded to the API.
                        "FileName" : "trailers/main/thumbnail.png",

                        # A plaintext description of what the image
represents.
                        "Description" : "The thumbnail for the trailer shown
" +
                                        "before the user clicks play"
                    },
                    {
                        "FileName" : "trailers/main/alt-img.png",
                        "Description" : "The image to show after the trailer
plays"
                    }
                ]
            }
        }
    }
    return trailer
```

# Related topics

- Create and manage submissions using Microsoft Store services

# Manage targeted offers using Store services

Article • 01/04/2023

If you create a *targeted offer* in the **Engage** > **Targeted offers** page for your app in Partner Center, use the *Microsoft Store targeted offers API* in your app's code to retrieve info that helps you implement the in-app experience for the targeted offer. For more information about targeted offers and how to create them in the dashboard, see Use targeted offers to maximize engagement and conversions.

The targeted offers API is a simple REST API that you can use to get the targeted offers that are available for the current user, based on whether or not the user is part of the customer segment for the targeted offer. To use this API in your app's code, follow these steps:

1. Get a Microsoft Account token for the current signed-in user of your app.
2. Get the targeted offers for the current user.
3. Implement the in-app purchase experience for the add-on that is associated with one of the targeted offers. For more information about implementing in-app purchases, see this article.

For a complete code example that demonstrates all of these steps, see the code example at the end of this article. The following sections provide more details about each step.

## Get a Microsoft Account token for the current user

In your app's code, get a Microsoft Account (MSA) token for the current signed-in user. You must pass this token in the `Authorization` request header for the Microsoft Store targeted offers API. This token is used by the Store to retrieve the targeted offers that are available for the current user.

To get the MSA token, use the WebAuthenticationCoreManager class to request a token using the scope `devcenter_implicit.basic,wl.basic`. The following example demonstrates how to do this. This example is an excerpt from the complete example, and it requires **using** statements that are provided in the complete example.

```
C#
```

```
private async Task<string> GetMicrosoftAccountTokenAsync()
{
    var msaProvider = await
WebAuthenticationCoreManager.FindAccountProviderAsync(
        "https://login.microsoft.com", "consumers");

    WebTokenRequest request = new WebTokenRequest(msaProvider,
"devcenter_implicit.basic,wl.basic");
    WebTokenRequestResult result = await
WebAuthenticationCoreManager.RequestTokenAsync(request);

    if (result.ResponseStatus == WebTokenRequestStatus.Success)
    {
        return result.ResponseData[0].Token;
    }
    else
    {
        return string.Empty;
    }
}
```

For more information about getting MSA tokens, see Web account manager.

# Get the targeted offers for the current user

After you have an MSA token for the current user, call the GET method of the `https://manage.devcenter.microsoft.com/v2.0/my/storeoffers/user` URI to get the available targeted offers for the current user. For more information about this REST method, see Get targeted offers.

This method returns the product IDs of the add-ons that that are associated with the targeted offers that are available for the current user. With this information, you can offer one or more of the targeted offers as an in-app purchase to the user.

The following example demonstrates how to get the targeted offers for the current user. This example is an excerpt from the complete example. It requires the Json.NET library from Newtonsoft and additional classes and **using** statements that are provided in the complete example.

C#

```
private async Task<List<TargetedOfferData>>
GetTargetedOffersForUserAsync(string msaToken)
{
    if (string.IsNullOrEmpty(msaToken))
    {
        System.Diagnostics.Debug.WriteLine("Microsoft Account token is null
```

```
or empty.");
        return null;
    }

    HttpClient httpClientGetOffers = new HttpClient();
    httpClientGetOffers.DefaultRequestHeaders.Authorization = new
AuthenticationHeaderValue("Bearer", msaToken);
    List<TargetedOfferData> availableOfferData = null;

    try
    {
        string getOffersResponse = await
httpClientGetOffers.GetStringAsync(new Uri(storeOffersUri));
        availableOfferData =

Newtonsoft.Json.JsonConvert.DeserializeObject<List<TargetedOfferData>>
(getOffersResponse);
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine(ex.Message);
    }

    return availableOfferData;
}
```

# Complete code example

The following code example demonstrates the following tasks:

- Get an MSA token for the current user.
- Get all of the targeted offers for the current user by using the Get targeted offers method.
- Purchase the add-on that is associated with a targeted offer.

This example requires the Json.NET library from Newtonsoft. The example uses this library to serialize and deserialize JSON-formatted data.

C#

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Net.Http;
using System.Net.Http.Headers;
using Newtonsoft.Json;
using Newtonsoft.Json.Linq;
using Windows.Services.Store;
```

```
using Windows.Security.Authentication.Web.Core;

namespace DocumenationExamples
{
    public class TargetedOffersExample
    {
        private const string storeOffersUri =
"https://manage.devcenter.microsoft.com/v2.0/my/storeoffers/user";
        private const string jsonMediaType = "application/json";
        private static string[] productKinds = { "Durable", "Consumable",
"UnmanagedConsumable" };
        private static StoreContext storeContext =
StoreContext.GetDefault();

        public async void DemonstrateTargetedOffers()
        {
            // Get the Microsoft Account token for the current user.
            string msaToken = await GetMicrosoftAccountTokenAsync();

            if (string.IsNullOrEmpty(msaToken))
            {
                System.Diagnostics.Debug.WriteLine("Microsoft Account token
could not be retrieved.");
                return;
            }

            // Get the targeted Store offers for the current user.
            List<TargetedOfferData> availableOfferData =
                await GetTargetedOffersForUserAsync(msaToken);

            if (availableOfferData == null || availableOfferData.Count == 0)
            {
                System.Diagnostics.Debug.WriteLine("There was an error
retrieving targeted offers," +
                    "or there are no targeted offers available for the
current user.");
                return;
            }

            // Get the product ID of the add-on that is associated with the
first available offer
            // in the response data.
            TargetedOfferData offerData = availableOfferData[0];
            string productId = offerData.Offers[0];

            // Get the Store ID of the add-on that has the matching product
ID, and then purchase the add-on.
            List<String> filterList = new List<string>(productKinds);
            StoreProductQueryResult queryResult = await
storeContext.GetAssociatedStoreProductsAsync(filterList);
            foreach (KeyValuePair<string, StoreProduct> result in
queryResult.Products)
            {
                if (result.Value.InAppOfferToken == productId)
                {
```

```
                    await PurchaseOfferAsync(result.Value.StoreId);
                    return;
                }
            }

            System.Diagnostics.Debug.WriteLine("No add-on with the specified
product ID could be found " +
                "for the current app.");
            return;
        }

        private async Task<string> GetMicrosoftAccountTokenAsync()
        {
            var msaProvider = await
WebAuthenticationCoreManager.FindAccountProviderAsync(
                "https://login.microsoft.com", "consumers");

            WebTokenRequest request = new WebTokenRequest(msaProvider,
"devcenter_implicit.basic,wl.basic");
            WebTokenRequestResult result = await
WebAuthenticationCoreManager.RequestTokenAsync(request);

            if (result.ResponseStatus == WebTokenRequestStatus.Success)
            {
                return result.ResponseData[0].Token;
            }
            else
            {
                return string.Empty;
            }
        }

        private async Task<List<TargetedOfferData>>
GetTargetedOffersForUserAsync(string msaToken)
        {
            if (string.IsNullOrEmpty(msaToken))
            {
                System.Diagnostics.Debug.WriteLine("Microsoft Account token
is null or empty.");
                return null;
            }

            HttpClient httpClientGetOffers = new HttpClient();
            httpClientGetOffers.DefaultRequestHeaders.Authorization = new
AuthenticationHeaderValue("Bearer", msaToken);
            List<TargetedOfferData> availableOfferData = null;

            try
            {
                string getOffersResponse = await
httpClientGetOffers.GetStringAsync(new Uri(storeOffersUri));
                availableOfferData =

Newtonsoft.Json.JsonConvert.DeserializeObject<List<TargetedOfferData>>
(getOffersResponse);
```

```
            }
            catch (Exception ex)
            {
                System.Diagnostics.Debug.WriteLine(ex.Message);
            }

            return availableOfferData;
        }

        private async Task PurchaseOfferAsync(string storeId)
        {
            if (string.IsNullOrEmpty(storeId))
            {
                System.Diagnostics.Debug.WriteLine("storeId is null or
empty.");
                return;
            }

            // Purchase the add-on for the current user. Typically, a game
or app would first show
            // a UI that prompts the user to buy the add-on; for simplicity,
this example
            // simply purchases the add-on.
            StorePurchaseResult result = await
storeContext.RequestPurchaseAsync(storeId);

            // Capture the error message for the purchase operation, if any.
            string extendedError = string.Empty;
            if (result.ExtendedError != null)
            {
                extendedError = result.ExtendedError.Message;
            }

            switch (result.Status)
            {
                case StorePurchaseStatus.AlreadyPurchased:
                    System.Diagnostics.Debug.WriteLine("The user has already
purchased the product.");
                    break;

                case StorePurchaseStatus.Succeeded:
                    System.Diagnostics.Debug.WriteLine("The purchase was
successful.");
                    break;

                case StorePurchaseStatus.NotPurchased:
                    System.Diagnostics.Debug.WriteLine("The purchase did not
complete. " +
                        "The user may have cancelled the purchase.
ExtendedError: " + extendedError);
                    break;

                case StorePurchaseStatus.NetworkError:
                    System.Diagnostics.Debug.WriteLine("The purchase was
unsuccessful due to a network error. " +
```

```
                        "ExtendedError: " + extendedError);
                    break;

                case StorePurchaseStatus.ServerError:
                    System.Diagnostics.Debug.WriteLine("The purchase was
unsuccessful due to a server error. " +
                        "ExtendedError: " + extendedError);
                    break;

                default:
                    System.Diagnostics.Debug.WriteLine("The purchase was
unsuccessful due to an unknown error. " +
                        "ExtendedError: " + extendedError);
                    break;
            }
        }
    }

    public class TargetedOfferData
    {
        [JsonProperty(PropertyName = "offers")]
        public IList<string> Offers { get; } = new List<string>();

        [JsonProperty(PropertyName = "trackingId")]
        public string TrackingId { get; set; }
    }
}
```

# Related topics

- Use targeted offers to maximize engagement and conversions
- Get targeted offers

# Get targeted offers

Article • 10/20/2022

Use this method to get the targeted offers that are available for the current user, based on whether or not the user is part of the customer segment for the targeted offer. For more information, see Manage targeted offers using Store services.

# Prerequisites

To use this method, you need to first obtain a Microsoft Account token for the current signed-in user of your app. You must pass this token in the `Authorization` request header for this method. This token is used by the Store to get targeted offers for the current user.

# Request

## Request syntax

| Method | Request URI |
| --- | --- |
| GET | `https://manage.devcenter.microsoft.com/v2.0/my/storeoffers/user` |

## Request header

| Header | Type | Description |
| --- | --- | --- |
| Authorization | string | Required. The Microsoft Account token for the current signed-in user of your app in the form **Bearer** <*token*>. |

## Request parameters

This method has no URI or query parameters.

## Request example

syntax

```
GET https://manage.devcenter.microsoft.com/v2.0/my/storeoffers/user HTTP/1.1
Authorization: Bearer <Microsoft Account token>
```

# Response

This method returns a JSON-formatted response body that contains an array of objects with the following fields. Each object in the array represents the targeted offers that are available for the specified user as part of a particular customer segment.

| Field | Type | Description |
|---|---|---|
| offers | array | An array of product IDs for the add-ons that are associated with the targeted offers that are available for the current user. These product IDs are specified in the **Targeted offers** page for your app in Partner Center. |
| trackingId | string | A GUID that you can optionally use to keep track of the targeted offer in your own code or services. |

## Example

The following example demonstrates an example JSON response body for this request.

JSON

```
[
  {
    "offers": [
      "10x gold coins",
      "100x gold coins"
    ],
    "trackingId": "5de5dd29-6dce-4e68-b45e-d8ee6c2cd203"
  }
]
```

# Related topics

- Manage targeted offers using Store services

# Manage product entitlements from a service

Article • 12/11/2024

If you have a catalog of apps and add-ons, you can use the *Microsoft Store collection API* and *Microsoft Store purchase API* to access entitlement information for these products from your services. An *entitlement* represents a customer's right to use an app or add-on that is published through the Microsoft Store.

These APIs consist of REST methods that are designed to be used by developers with add-on catalogs that are supported by cross-platform services. These APIs enable you to do the following:

- Microsoft Store collection API: Query for products owned by a user and report a consumable product as fulfilled.
- Microsoft Store purchase API: Grant a free product to a user, get subscriptions for a user, and change the billing state of a subscription for a user.

> **Note**
>
> The Microsoft Store collection API and purchase API use The Microsoft identity platform (Entra ID) authentication to access customer ownership information. To use these APIs, you (or your organization) must have an Entra ID tenant and you must have Global administrator permission for the tenant. If you already use Microsoft 365 or other business services from Microsoft, you already have Entra ID tenant.

## The Microsoft.StoreServices library

To help streamline the authentication flow and calling the Microsoft Store Services, review the Microsoft.StoreServices project and sample on Github. The Microsoft.StoreServices library will help manage the authentication keys and provides wrapper API's to call into the Microsoft Store Services for managing products. The sample project highlights how a service can use the Microsoft.StoreServices library, example logic for managing consumable products, reconciling refunded purchases, renew expired credentials, and more. A step-by-step configuration guide is included with the sample to setup the sample service on your PC or through Azure.

- Microsoft.StoreServices library (GitHub)

- Microsoft.StoreServices Sample (GitHub)

# Overview

The following steps describe the end-to-end process for using the Microsoft Store collection API and purchase API:

1. Configure an application in Entra ID.
2. Associate your Entra ID application ID with your app in Partner Center.
3. In your service, create Entra ID access tokens that represent your publisher identity.
4. In your client Windows app, create a Microsoft Store ID key that represents the identity of the current user, and pass this key back to your service.

This end-to-end process involves two software components that perform different tasks:

- **Your service**. This is an application that runs securely in the context of your business environment, and it can be implemented using any development platform you choose. Your service is responsible for creating the Entra ID access tokens needed for the scenario and for calling the REST URIs for the Microsoft Store collection API and purchase API.
- **Your client Windows app**. This is the app for which you want to access and manage customer entitlement information (including add-ons for the app). This app is responsible for creating the Microsoft Store ID keys you need to call the Microsoft Store collection API and purchase API from your service.

# Step 1: Configure an application in Entra ID

Before you can use the Microsoft Store collection API or purchase API, you must create an Entra ID Web application, retrieve the tenant ID and application ID for the application, and generate a key. The Entra ID Web application represents the service from which you want to call the Microsoft Store collection API or purchase API. You need the tenant ID, application ID and key to generate Entra ID access tokens that you need to call the API.

1. If you haven't done so already, follow the instructions in Quickstart: Register an application with the Microsoft identity platform to register a **Web app / API** application with Entra ID.

   > **Note**

When you register your application, you must choose **Web app / API** as the application type so that you can retrieve a key (also called a *client secret*) for your application. In order to call the Microsoft Store collection API or purchase API, you must provide a client secret when you request an access token from Entra ID in a later step.

2. In the Azure Management Portal, navigate to **Microsoft Entra ID**. Select your tenant, click **App registrations** in the left navigation pane under Manage, and then select your application.

3. You are taken to the application's main registration page. On this page, copy the **Application ID** value for use later.

4. Create a key that you will need later (this is all called a *client secret*). In the left pane, click **Settings** and then **Keys**. On this page, complete the steps to create a key. Copy this key for later use.

## Step 2: Associate your Entra ID application ID with your client app in Partner Center

Before you can use the Microsoft Store collection API or purchase API to configure the ownership and purchases for your app or add-on, you must associate your Entra application ID with the app (or the app that contains the add-on) in Partner Center.

> **Note**
>
> You only need to perform this task one time. After you have your tenant ID, application ID and client secret, you can reuse these values any time you need to create a new Entra ID access token.

1. Sign in to Partner Center and select your app.
2. Go to the **Services** > **Product collections and purchases** page and enter your Entra application ID into one of the available **Client ID** fields.

## Step 3: Create Entra ID access tokens

Before you can retrieve a Microsoft Store ID key or call the Microsoft Store collection API or purchase API, your service must create several different Entra ID access tokens that represent your publisher identity. Each token will be used with a different API. The lifetime of each token is 60 minutes, and you can refresh them after they expire.

> **Important**
>
> Create Entra ID access tokens only in the context of your service, not in your app. Your client secret could be compromised if it is sent to your app.

## Understanding the different tokens and audience URIs

Depending on which methods you want to call in the Microsoft Store collection API or purchase API, you must create either two or three different tokens. Each access token is associated with a different audience URI.

- In all cases, you must create a token with the `https://onestore.microsoft.com` audience URI. In a later step, you will pass this token to the **Authorization** header of methods in the Microsoft Store collection API or purchase API.

  > **Important**
  >
  > Use the `https://onestore.microsoft.com` audience only with access tokens that are stored securely within your service. Exposing access tokens with this audience outside your service could make your service vulnerable to replay attacks.

- If you want to call a method in the Microsoft Store collection API to query for products owned by a user or report a consumable product as fulfilled, you must also create a token with the `https://onestore.microsoft.com/b2b/keys/create/collections` audience URI. In a later step, you will pass this token to a client method in the Windows SDK to request a Microsoft Store ID key that you can use with the Microsoft Store collection API.

- If you want to call a method in the Microsoft Store purchase API to grant a free product to a user, get subscriptions for a user, or change the billing state of a subscription for a user, you must also create a token with the `https://onestore.microsoft.com/b2b/keys/create/purchase` audience URI. In a later step, you will pass this token to a client method in the Windows SDK to request a Microsoft Store ID key that you can use with the Microsoft Store purchase API.

## Create the tokens

To create the access tokens, use the OAuth 2.0 API in your service by following the instructions in Service to Service Calls Using Client Credentials to send an HTTP POST to the `https://login.microsoftonline.com/<tenant_id>/oauth2/token` endpoint. Here is a sample request.

syntax

```
POST https://login.microsoftonline.com/<tenant_id>/oauth2/token HTTP/1.1
Host: login.microsoftonline.com
Content-Type: application/x-www-form-urlencoded; charset=utf-8

grant_type=client_credentials
&client_id=<your_client_id>
&client_secret=<your_client_secret>
&resource=https://onestore.microsoft.com
```

For each token, specify the following parameter data:

- For the *client_id* and *client_secret* parameters, specify the application ID and the client secret for your application that you retrieved from the Azure Management Portal. Both of these parameters are required in order to create an access token with the level of authentication required by the Microsoft Store collection API or purchase API.
- For the *resource* parameter, specify one of the audience URIs listed in the previous section, depending on the type of access token you are creating.

After your access token expires, you can refresh it by following the instructions here. For more details about the structure of an access token, see Supported Token and Claim Types.

# Step 4: Create a Microsoft Store ID key

Before you can call any method in the Microsoft Store collection API or purchase API, your app must create a Microsoft Store ID key and send it to your service. This key is a JSON Web Token (JWT) that represents the identity of the user whose product ownership information you want to access. For more information about the claims in this key, see Claims in a Microsoft Store ID key.

Currently, the only way to create a Microsoft Store ID key is by calling a Universal Windows Platform (UWP) API from client code in your app. The generated key represents the identity of the user who is currently signed in to the Microsoft Store on the device.

> **Note**
>
> Each Microsoft Store ID key is valid for 30 days. Before the key expires, you can renew the key. We recommend that you renew your Microsoft Store ID keys rather than creating new ones.

## To create a Microsoft Store ID key for the Microsoft Store collection API

Follow these steps to create a Microsoft Store ID key that you can use with the Microsoft Store collection API to query for products owned by a user or report a consumable product as fulfilled.

1. Pass the Entra ID access token that has the audience URI value `https://onestore.microsoft.com/b2b/keys/create/collections` from your service to your client app. This is one of the tokens you created earlier in step 3.

2. In your app code, call one of these methods to retrieve a Microsoft Store ID key:

- If your app uses the StoreContext class in the Windows.Services.Store namespace to manage in-app purchases, use the StoreContext.GetCustomerCollectionsIdAsync method.

- If your app uses the CurrentApp class in the Windows.ApplicationModel.Store namespace to manage in-app purchases, use the CurrentApp.GetCustomerCollectionsIdAsync method.

  Pass your Entra ID access token to the *serviceTicket* parameter of the method. If you maintain anonymous user IDs in the context of services that you manage as the publisher of the current app, you can also pass a user ID to the *publisherUserId* parameter to associate the current user with the new Microsoft Store ID key (the user ID will be embedded in the key). Otherwise, if you don't need to associate a user ID with the Microsoft Store ID key, you can pass any string value to the *publisherUserId* parameter.

3. After your app successfully creates a Microsoft Store ID key, pass the key back to your service.

## To create a Microsoft Store ID key for the Microsoft Store purchase API

Follow these steps to create a Microsoft Store ID key that you can use with the Microsoft Store purchase API to grant a free product to a user, get subscriptions for a user, or change the billing state of a subscription for a user.

1. Pass the Entra ID access token that has the audience URI value `https://onestore.microsoft.com/b2b/keys/create/purchase` from your service to your client app. This is one of the tokens you created earlier in step 3.

2. In your app code, call one of these methods to retrieve a Microsoft Store ID key:

- If your app uses the StoreContext class in the Windows.Services.Store namespace to manage in-app purchases, use the StoreContext.GetCustomerPurchaseIdAsync method.

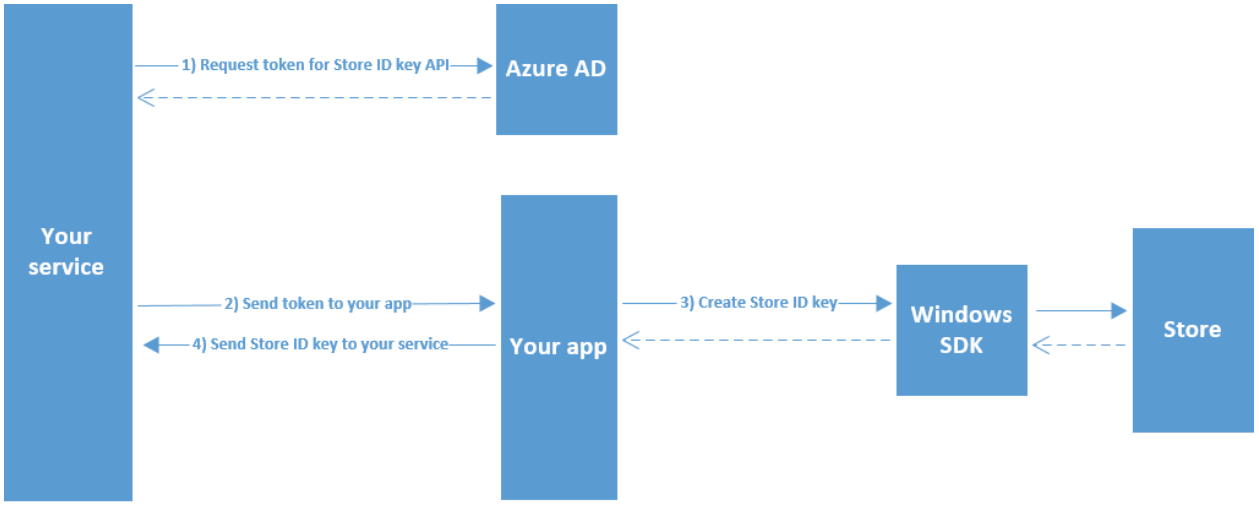
- If your app uses the CurrentApp class in the Windows.ApplicationModel.Store namespace to manage in-app purchases, use the CurrentApp.GetCustomerPurchaseIdAsync method.

  Pass your Entra ID access token to the *serviceTicket* parameter of the method. If you maintain anonymous user IDs in the context of services that you manage as the publisher of the current app, you can also pass a user ID to the *publisherUserId* parameter to associate the current user with the new Microsoft Store ID key (the user ID will be embedded in the key). Otherwise, if you don't need to associate a user ID with the Microsoft Store ID key, you can pass any string value to the *publisherUserId* parameter.

3. After your app successfully creates a Microsoft Store ID key, pass the key back to your service.

## Diagram

The following diagram illustrates the process of creating a Microsoft Store ID key.



# Claims in a Microsoft Store ID key

A Microsoft Store ID key is a JSON Web Token (JWT) that represents the identity of the user whose product ownership information you want to access. When decoded using Base64, a Microsoft Store ID key contains the following claims.

- `iat`: Identifies the time at which the key was issued. This claim can be used to determine the age of the token. This value is expressed as epoch time.
- `iss`: Identifies the issuer. This has the same value as the `aud` claim.
- `aud`: Identifies the audience. Must be one of the following values: `https://collections.mp.microsoft.com/v6.0/keys` or `https://purchase.mp.microsoft.com/v6.0/keys`.
- `exp`: Identifies the expiration time on or after which the key will no longer be accepted for processing anything except for renewing keys. The value of this claim is expressed as epoch time.
- `nbf`: Identifies the time at which the token will be accepted for processing. The value of this claim is expressed as epoch time.
- `http://schemas.microsoft.com/marketplace/2015/08/claims/key/clientId`: The client ID that identifies the developer.
- `http://schemas.microsoft.com/marketplace/2015/08/claims/key/payload`: An opaque payload (encrypted and Base64 encoded) that contains information that is intended only for use by Microsoft Store services.
- `http://schemas.microsoft.com/marketplace/2015/08/claims/key/userId`: A user ID that identifies the current user in the context of your services. This is the same value you pass into the optional *publisherUserId* parameter of the method you use to create the key.
- `http://schemas.microsoft.com/marketplace/2015/08/claims/key/refreshUri`: The URI that you can use to renew the key.

Here is an example of a decoded Microsoft Store ID key header.

JSON

```
{
    "typ":"JWT",
    "alg":"RS256",
    "x5t":"agA_pgJ7Twx_Ex2_rEeQ2o5fZ5g"
}
```

Here is an example of a decoded Microsoft Store ID key claim set.

JSON

```
{
    "http://schemas.microsoft.com/marketplace/2015/08/claims/key/clientId":
"1d5773695a3b44928227393bfef1e13d",
    "http://schemas.microsoft.com/marketplace/2015/08/claims/key/payload":
"ZdcOq0/N2rjytCRzCHSqnfczv3f0343wfSydx7hghfu0snWzMqyoAGy5DSJ5rMSsKoQFAccs1iN
lwlGrX+/eIwh/VlUhLrncyP8c18mNAzAGK+lTAd2oiMQWRRAZxPwGrJrwiq2fTq5NOVDnQS9Za6/
GdRjeiQrv6c0x+WNKxSQ7LV/uH1x+IEhYVtDu53GiXIwekltwaV6EkQGphYy7tbNsW2GqxgcoLLM
UVOsQjI+FYBA3MdQpalV/aFN4UrJDkMWJBnmz3vrxBNGEApLWTS4Bd3cMswXsV9m+VhOEfnv+6Pr
L2jq8OZFoF3FUUpY8Fet2DfFr6xjZs3CBS1095J2yyNFWKBZxAXXNjn+zkvqqiVRjjkjNajhuaNK
Jk4MGHfk2rZiMy/aosyaEpCyncdisHVSx/S4JwIuxTnfnlY24vS0OXy7mFiZjjB8qL03cLsBXM4u
tCyXSIggb90GAx0+EFlVoJD7+ZKlm1M90xO/QSMDlrzFyuqcXXDBOnt7rPynPTrOZLVF+ODI5HhW
EqArkVnc5MYnrZD06YEwClmTDkHQcxCvU+XUEvTbEk69qR2sfnuXV4cJRRWseUTfYoGyuxkQ2eWA
AI1BXGxYECIaAnWF0W6ThweL5ZZDdadW9Ug5U3fZd4WxiDlB/EZ3aTy8kYXTW4Uo0adTkCmdLibw
=",
    "http://schemas.microsoft.com/marketplace/2015/08/claims/key/userId":
"infusQMLaYCrgtC0d/SZWoPB4FqLEwHXgZFuMJ6TuTY=",

"http://schemas.microsoft.com/marketplace/2015/08/claims/key/refreshUri":
"https://collections.mp.microsoft.com/v6.0/b2b/keys/renew",
    "iat": 1733526889,
    "iss": "https://collections.mp.microsoft.com/v6.0/keys",
    "aud": "https://collections.mp.microsoft.com/v6.0/keys",
    "exp": 1733523289,
    "nbf": 1736118889
}
```


## Related topics

- Query for products
- Report consumable products as fulfilled
- Grant free products
- Get subscriptions for a user
- Change the billing state of a subscription for a user
- Renew a Microsoft Store ID key
- Integrating Applications with the Microsoft identity platform
- ID tokens in the Microsoft identity platform
- Microsoft.StoreServices library (GitHub)

## Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

# Query for products

Article • 10/20/2022

Use this method in the Microsoft Store collection API to get all the products that a customer owns for apps that are associated with your Azure AD client ID. You can scope your query to a particular product, or use other filters.

This method is designed to be called by your service in response to a message from your app. Your service should not regularly poll for all users on a schedule.

The Microsoft.StoreServices library provides the functionality of this method through the StoreServicesClient.CollectionsQueryAsync API.

## Prerequisites

To use this method, you will need:

- An Azure AD access token that has the audience URI value `https://onestore.microsoft.com`.
- A Microsoft Store ID key that represents the identity of the user whose products you want to get.

For more information, see Manage product entitlements from a service.

## Request

### Request syntax

Expand table

| Method | Request URI |
| --- | --- |
| POST | `https://collections.mp.microsoft.com/v6.0/collections/query` |

### Request header

Expand table

| Header | Type | Description |
|---|---|---|
| Authorization | string | Required. The Azure AD access token in the form **Bearer** *<token>*. |
| Host | string | Must be set to the value **collections.mp.microsoft.com**. |
| Content-Length | number | The length of the request body. |
| Content-Type | string | Specifies the request and response type. Currently, the only supported value is **application/json**. |

## Request body

Expand table

| Parameter | Type | Description | Required |
|---|---|---|---|
| beneficiaries | list<UserIdentity> | A list of UserIdentity objects that represent the users being queried for products. For more information, see the table below. | Yes |
| continuationToken | string | If there are multiple sets of products, the response body returns a continuation token when the page limit is reached. Provide that continuation token here in subsequent calls to retrieve remaining products. | No |
| maxPageSize | number | The maximum number of products to return in one response. The default and maximum value is 100. | No |
| modifiedAfter | datetime | If specified, the service only returns products that have been modified after this date. | No |
| parentProductId | string | If specified, the service only returns add-ons that correspond to the specified app. | No |
| productSkuIds | list<ProductSkuId> | If specified, the service only returns products applicable to the provided product/SKU pairs. For more information, see the table below. | No |
| productTypes | list<string> | Specifies which products types to return in the query results. Supported product types are **Application**, **Durable**, **Game**, and **UnmanagedConsumable**. | Yes |

| Parameter | Type | Description | Required |
|---|---|---|---|
| validityType | string | When set to **All**, all products for a user will be returned, including expired items. When set to **Valid**, only products that are valid at this point in time are returned (that is, they have an active status, start date < now, and end date is > now). | No |

The UserIdentity object contains the following parameters.

Expand table

| Parameter | Type | Description | Required |
|---|---|---|---|
| identityType | string | Specify the string value **b2b**. | Yes |
| identityValue | string | The Microsoft Store ID key that represents the identity of the user for whom you want to query products. | Yes |
| localTicketReference | string | The requested identifier for the returned products. Returned items in the response body will have a matching *localTicketReference*. We recommend that you use the same value as the *userId* claim in the Microsoft Store ID key. | Yes |

The ProductSkuId object contains the following parameters.

Expand table

| Parameter | Type | Description | Required |
|---|---|---|---|
| productId | string | The Store ID for a product in the Microsoft Store catalog. An example Store ID for a product is 9NBLGGH42CFD. | Yes |
| skuId | string | The Store ID for a product's SKU in the Microsoft Store catalog. An example Store ID for a SKU is 0010. | Yes |

## Request example

syntax

```
POST https://collections.mp.microsoft.com/v6.0/collections/query HTTP/1.1
Authorization: Bearer eyJ0eXAiOiJKV1Q…….
Host: collections.mp.microsoft.com
Content-Length: 2531
Content-Type: application/json
```

```
{
  "maxPageSize": 100,
  "beneficiaries": [
    {
      "localTicketReference": "1055521810674918",
      "identityValue": "eyJ0eXAiOiJ……",
      "identityType": "b2b"
    }
  ],
  "modifiedAfter": "\/Date(-62135568000000)\/",
  "productSkuIds": [
    {
      "productId": "9NBLGGH5WVP6",
      "skuId": "0010"
    }
  ],
  "productTypes": [
    "UnmanagedConsumable"
  ],
  "validityType": "All"
}
```

# Response

## Response body

Expand table

| Parameter | Type | Description | Required |
|---|---|---|---|
| continuationToken | string | If there are multiple sets of products, this token is returned when the page limit is reached. You can specify this continuation token in subsequent calls to retrieve remaining products. | No |
| items | CollectionItemContractV6 | An array of products for the specified user. For more information, see the table below. | No |

The CollectionItemContractV6 object contains the following parameters.

Expand table

| Parameter | Type | Description | Required |
|---|---|---|---|
| acquiredDate | datetime | The date on which the user acquired the item. | Yes |
| campaignId | string | The campaign ID that was provided at purchase time for this item. | No |
| devOfferId | string | The offer ID from an in-app purchase. | No |
| endDate | datetime | The end date of the item. | Yes |
| fulfillmentData | list<string> | N/A | No |
| inAppOfferToken | string | The developer-specified product ID string that is assigned to the item in Partner Center. An example product ID is *product123*. | No |
| itemId | string | An ID that identifies this collection item from other items the user owns. This ID is unique per product. | Yes |
| localTicketReference | string | The ID of the previously supplied *localTicketReference* in the request body. | Yes |
| modifiedDate | datetime | The date this item was last modified. | Yes |
| orderId | string | If present, the order ID of which this item was obtained. | No |
| orderLineItemId | string | If present, the line item of the particular order for which this item was obtained. | No |
| ownershipType | string | The string *OwnedByBeneficiary*. | Yes |
| productId | string | The Store ID for the product in the Microsoft Store catalog. An example Store ID for a product is 9NBLGGH42CFD. | Yes |
| productType | string | One of the following product types: **Application**, **Durable**, and **UnmanagedConsumable**. | Yes |
| purchasedCountry | string | N/A | No |
| purchaser | IdentityContractV6 | If present, this represents the identity of the purchaser of the item. See the details for this object below. | No |
| quantity | number | The quantity of the item. Currently, this will always be 1. | No |

| Parameter | Type | Description | Required |
|---|---|---|---|
| skuId | string | The Store ID for the product's SKU in the Microsoft Store catalog. An example Store ID for a SKU is 0010. | Yes |
| skuType | string | Type of the SKU. Possible values include **Trial**, **Full**, and **Rental**. | Yes |
| startDate | datetime | The date that the item starts being valid. | Yes |
| status | string | The status of the item. Possible values include **Active**, **Expired**, **Revoked**, and **Banned**. | Yes |
| tags | list<string> | N/A | Yes |
| transactionId | guid | The transaction ID as a result of the purchase of this item. Can be used for reporting an item as fulfilled. | Yes |

The IdentityContractV6 object contains the following parameters.

Expand table

| Parameter | Type | Description | Required |
|---|---|---|---|
| identityType | string | Contains the value *pub*. | Yes |
| identityValue | string | The string value of the *publisherUserId* from the specified Microsoft Store ID key. | Yes |

## Response example

syntax

```
HTTP/1.1 200 OK
Content-Length: 7241
Content-Type: application/json
MS-CorrelationId: aaaa0000-bb11-2222-33cc-444444dddddd
MS-RequestId: a9988cf9-652b-4791-beba-b0e732121a12
MS-CV: xu2HW6SrSkyfHyFh.0.1
MS-ServerId: 020022359
Date: Tue, 22 Sep 2015 20:28:18 GMT

{
  "items" : [
    {
      "acquiredDate" : "2015-09-22T19:22:51.2068724+00:00",
      "devOfferId" : "f9587c53-540a-498b-a281-8a349491ed47",
```

```
      "endDate" : "9999-12-31T23:59:59.9999999+00:00",
      "fulfillmentData" : [],
      "inAppOfferToken" : "consumable2",
      "itemId" : "4b8fbb13127a41f299270ea668681c1d",
      "localTicketReference" : "1055521810674918",
      "modifiedDate" : "2015-09-22T19:22:51.2513155+00:00",
      "orderId" : "4ba5960d-4ec6-4a81-ac20-aafce02ddf31",
      "ownershipType" : "OwnedByBeneficiary",
      "productId" : "9NBLGGH5WVP6",
      "productType" : "UnmanagedConsumable",
      "purchaser" : {
        "identityType" : "pub",
        "identityValue" : "user123"
      },
      "skuId" : "0010",
      "skuType" : "Full",
      "startDate" : "2015-09-22T19:22:51.2068724+00:00",
      "status" : "Active",
      "tags" : [],
      "transactionId" : "4ba5960d-4ec6-4a81-ac20-aafce02ddf31"
    }
  ]
}
```


# Related topics

- Manage product entitlements from a service
- Report consumable products as fulfilled
- Grant free products
- Renew a Microsoft Store ID key
- Microsoft.StoreServices library (GitHub)

# Feedback

# Report consumable products as fulfilled

Article • 10/20/2022

Use this method in the Microsoft Store collection API to report a consumable product as fulfilled for a given customer. Before a user can repurchase a consumable product, your app or service must report the consumable product as fulfilled for that user.

There are two ways you can use this method to report a consumable product as fulfilled:

- Provide the item ID of the consumable (as returned in the **itemId** parameter of a query for products), and a unique tracking ID that you provide. If the same tracking ID is used for multiple tries, the same result will be returned even if the item is already consumed. If you aren't certain if a consume request was successful, your service should resubmit consume requests with the same tracking ID. The tracking ID will always be tied to that consume request and can be resubmitted indefinitely.
- Provide the product ID (as returned in the **productId** parameter of a query for products) and a transaction ID that is obtained from one of the sources listed in the description for the **transactionId** parameter in the request body section below.

The Microsoft.StoreServices library provides the functionality of this method through the StoreServicesClient.CollectionsConsumeAsync API.

## Prerequisites

To use this method, you will need:

- An Azure AD access token that has the audience URI value `https://onestore.microsoft.com`.
- A Microsoft Store ID key that represents the identity of the user for whom you want to report a consumable product as fulfilled.

For more information, see Manage product entitlements from a service.

## Request

### Request syntax

Expand table

| Method | Request URI |
| --- | --- |
| POST | `https://collections.mp.microsoft.com/v6.0/collections/consume` |

## Request header

Expand table

| Header | Type | Description |
| --- | --- | --- |
| Authorization | string | Required. The Azure AD access token in the form **Bearer** <*token*>. |
| Host | string | Must be set to the value **collections.mp.microsoft.com**. |
| Content-Length | number | The length of the request body. |
| Content-Type | string | Specifies the request and response type. Currently, the only supported value is **application/json**. |

## Request body

Expand table

| Parameter | Type | Description | Required |
| --- | --- | --- | --- |
| beneficiary | UserIdentity | The user for which this item is being consumed. For more information, see the following table. | Yes |
| itemId | string | The *itemId* value returned by a query for products. Use this parameter with *trackingId* | No |
| trackingId | guid | A unique tracking ID provided by developer. Use this parameter with *itemId*. | No |
| productId | string | The *productId* value returned by a query for products. Use this parameter with *transactionId* | No |
| transactionId | guid | A transaction ID value that is obtained from one of the following sources. Use this parameter with *productId*.<br>• The TransactionID property of the PurchaseResults class.<br>• The app or product receipt that is returned by RequestProductPurchaseAsync, RequestAppPurchaseAsync, or GetAppReceiptAsync. | No |

| Parameter | Type | Description | Required |
| --- | --- | --- | --- |
| | | • The *transactionId* parameter returned by a query for products. | |

The UserIdentity object contains the following parameters.

Expand table

| Parameter | Type | Description | Required |
| --- | --- | --- | --- |
| identityType | string | Specify the string value **b2b**. | Yes |
| identityValue | string | The Microsoft Store ID key that represents the identity of the user for whom you want to report a consumable product as fulfilled. | Yes |
| localTicketReference | string | The requested identifier for the returned response. We recommend that you use the same value as the *userId* claim in the Microsoft Store ID key. | Yes |

## Request examples

The following example uses *itemId* and *trackingId*.

syntax

```
POST https://collections.mp.microsoft.com/v6.0/collections/consume HTTP/1.1
Authorization: Bearer eyJ0eXAiOiJKV1…..
Host: collections.mp.microsoft.com
Content-Length: 2050
Content-Type: application/json

{
    "beneficiary": {
        "localTicketReference": "testreference",
        "identityValue": "eyJ0eXAiOi…..",
        "identityType": "b2b"
    },
    "itemId": "44c26106-4979-457b-af34-609ae97a084f",
    "trackingId": "44db79ca-e31d-49e9-8896-fa5c7f892b40"
}
```

The following example uses *productId* and *transactionId*.

syntax

```
POST https://collections.mp.microsoft.com/v6.0/collections/consume HTTP/1.1
Authorization: Bearer eyJ0eXAiOiJKV1……
Content-Length: 1880
Content-Type: application/json
Host: collections.md.mp.microsoft.com

{
    "beneficiary" : {
        "localTicketReference" : "testReference",
        "identityValue" : "eyJ0eXAiOiJ…..",
        "identitytype" : "b2b"
    },
    "productId" : "9NBLGGH5WVP6",
    "transactionId" : "08a14c7c-1892-49fc-9135-190ca4f10490"
}
```

# Response

No content will be returned if the consumption was executed successfully.

## Response example

syntax

```
HTTP/1.1 204 No Content
Content-Length: 0
MS-CorrelationId: aaaa0000-bb11-2222-33cc-444444dddddd
MS-RequestId: e488cd0a-9fb6-4c2c-bb77-e5100d3c15b1
MS-CV: 5.1
MS-ServerId: 030011326
Date: Tue, 22 Sep 2015 20:40:55 GMT
```

# Error codes

Expand table

| Code | Error | Inner error code | Description |
|---|---|---|---|
| 401 | Unauthorized | AuthenticationTokenInvalid | The Azure AD access token is invalid. In some cases the details of the ServiceError will contain more information, such as when the token is expired or the *appid* claim is missing. |

| Code | Error | Inner error code | Description |
| --- | --- | --- | --- |
| 401 | Unauthorized | PartnerAadTicketRequired | An Azure AD access token was not passed to the service in the authorization header. |
| 401 | Unauthorized | InconsistentClientId | The *clientId* claim in the Microsoft Store ID key in the request body and the *appid* claim in the Azure AD access token in the authorization header do not match. |

## Related topics

- Manage product entitlements from a service
- Query for products
- Grant free products
- Renew a Microsoft Store ID key
- Microsoft.StoreServices library (GitHub)

## Feedback

# Grant free products

Article • 10/20/2022

Use this method in the Microsoft Store purchase API to grant a free app or add-on (also known as in-app product or IAP) to a given user.

Currently, you can only grant free products. If your service attempts to use this method to grant a product that is not free, this method will return an error.

## Prerequisites

To use this method, you will need:

- An Azure AD access token that has the audience URI value `https://onestore.microsoft.com`.
- A Microsoft Store ID key that represents the identity of the user for whom you want to grant a free product.

For more information, see Manage product entitlements from a service.

## Request

### Request syntax

Expand table

| Method | Request URI |
|---|---|
| POST | `https://purchase.mp.microsoft.com/v6.0/purchases/grant` |

### Request header

Expand table

| Header | Type | Description |
|---|---|---|
| Authorization | string | Required. The Azure AD access token in the form **Bearer** <*token*>. |
| Host | string | Must be set to the value **purchase.mp.microsoft.com**. |

| Header | Type | Description |
| --- | --- | --- |
| Content-Length | number | The length of the request body. |
| Content-Type | string | Specifies the request and response type. Currently, the only supported value is **application/json**. |

## Request body

Expand table

| Parameter | Type | Description | Required |
| --- | --- | --- | --- |
| availabilityId | string | The availability ID of the product to be granted from the Microsoft Store catalog. | Yes |
| b2bKey | string | The Microsoft Store ID key that represents the identity of the user for whom you want to grant a product. | Yes |
| devOfferId | string | A developer-specified offer ID that will appear in the Collection item after purchase. | |
| language | string | The language of the user. | Yes |
| market | string | The market of the user. | Yes |
| orderId | guid | A GUID generated for the order. This value be unique for the user, but it is not required to be unique across all orders. | Yes |
| productId | string | The Store ID for the product in the Microsoft Store catalog. An example Store ID for a product is 9NBLGGH42CFD. | Yes |
| quantity | int | The quantity to purchase. Currently, the only supported value is 1. If not specified, the default is 1. | No |
| skuId | string | The Store ID for the product's SKU in the Microsoft Store catalog. An example Store ID for a SKU is 0010. | Yes |

## Request example

syntax

```
POST https://purchase.mp.microsoft.com/v6.0/purchases/grant HTTP/1.1
Authorization: Bearer eyJ0eXAiOiJK……
Content-Length: 1863
Content-Type: application/json
```

```
{
    "b2bKey" : "eyJ0eXAiOiJK……",
    "availabilityId" : "9RT7C09D5J3W",
    "productId" : "9NBLGGH5WVP6",
    "skuId" : "0010",
    "language" : "en-us",
    "market" : "us",
    "orderId" : "3eea1529-611e-4aee-915c-345494e4ee76",
}
```

# Response

## Response body

Expand table

| Parameter | Type | Description | Required |
|---|---|---|---|
| clientContext | ClientContextV6 | Client contextual information for this order. This will be assigned to the *clientID* value from the Azure AD token. | Yes |
| createdtime | datetimeoffset | The time the order was created. | Yes |
| currencyCode | string | Currency code for *totalAmount* and *totalTaxAmount*. N/A for free items. | Yes |
| friendlyName | string | The friendly name for the order. N/A for orders made using the Microsoft Store purchase API. | Yes |
| isPIRequired | boolean | Indicates whether a payment instrument (PI) is required as part of the purchase order. | Yes |
| language | string | The language ID for the order (for example, “en”). | Yes |
| market | string | The market ID for the order (for example, “US”). | Yes |
| orderId | string | An ID that identifies the order for a particular user. | Yes |

| Parameter | Type | Description | Required |
|---|---|---|---|
| orderLineItems | list<OrderLineItemV6> | The list of line items for the order. Typically there is 1 line item per order. | Yes |
| orderState | string | The state of the order. The valid states are **Editing**, **CheckingOut**, **Pending**, **Purchased**, **Refunded**, **ChargedBack**, and **Cancelled**. | Yes |
| orderValidityEndTime | string | The last time the order pricing is valid before it is submitted. N/A for free apps. | Yes |
| orderValidityStartTime | string | The first time the order pricing is valid before it is submitted. N/A for free apps. | Yes |
| purchaser | IdentityV6 | An object that describes the identity of the purchaser. | Yes |
| totalAmount | decimal | The total purchase amount of all items in the order including tax. | Yes |
| totalAmountBeforeTax | decimal | Total purchase amount of all items in the order before tax. | Yes |
| totalChargedToCsvTopOffPI | decimal | If using a separate payment instrument and stored value (CSV), the amount charged to CSV. | Yes |
| totalTaxAmount | decimal | The total amount of tax for all line items. | Yes |

The ClientContext object contains the following parameters.

Expand table

| Parameter | Type | Description | Required |
|---|---|---|---|
| client | string | The client ID that created the order. | No |

The OrderLineItemV6 object contains the following parameters.

Expand table

| Parameter | Type | Description | Required |
| --- | --- | --- | --- |
| agent | IdentityV6 | The agent that last edited the line item. For more information about this object, see the table below. | No |
| availabilityId | string | The availability ID of the product to be purchased from the Microsoft Store catalog. | Yes |
| beneficiary | IdentityV6 | The identity of the beneficiary of the order. | No |
| billingState | string | The billing state of the order. This is set to **Charged** when completed. | No |
| campaignId | string | The campaign ID for this order. | No |
| currencyCode | string | The currency code used for price details. | Yes |
| description | string | A localized description of the line item. | Yes |
| devofferId | string | The offer ID for the particular order, if present. | No |
| fulfillmentDate | datetimeoffset | The date the fulfillment occurred. | No |
| fulfillmentState | string | The state of the fulfillment of this item. This is set to **Fulfilled** when completed. | No |
| isPIRequired | boolean | Indicates whether a payment instrument is required for this line item. | Yes |
| isTaxIncluded | boolean | Indicated whether tax is included in the pricing details of the item. | Yes |
| legacyBillingOrderId | string | The legacy billing ID. | No |
| lineItemId | string | The line item ID for the item in this order. | Yes |
| listPrice | decimal | The list price of the item in this order. | Yes |
| productId | string | The Store ID for the product that represents the line item in the Microsoft Store catalog. An example Store ID for a product is 9NBLGGH42CFD. | Yes |
| productType | string | The type of the product. The supported values are **Durable**, **Application**, and **UnmanagedConsumable**. | Yes |

| Parameter | Type | Description | Required |
|---|---|---|---|
| quantity | int | The quantity of the item ordered. | Yes |
| retailPrice | decimal | The retail price of the item ordered. | Yes |
| revenueRecognitionState | string | The revenue recognition state. | Yes |
| skuId | string | The Store ID for the SKU of the line item in the Microsoft Store catalog. An example Store ID for a SKU is 0010. | Yes |
| taxAmount | decimal | The tax amount for the line item. | Yes |
| taxType | string | The tax type for the applicable taxes. | Yes |
| Title | string | The localized title of the line item. | Yes |
| totalAmount | decimal | The total purchase amount of the line item including tax. | Yes |

The IdentityV6 object contains the following parameters.

Expand table

| Parameter | Type | Description | Required |
|---|---|---|---|
| identityType | string | Contains the value "**pub**". | Yes |
| identityValue | string | The string value of the *publisherUserId* from the specified Microsoft Store ID key. | Yes |

## Response example

syntax

```
Content-Length: 1203
Content-Type: application/json
MS-CorrelationId: aaaa0000-bb11-2222-33cc-444444dddddd
MS-RequestId: c1bc832c-f742-47e4-a76c-cf061402f698
MS-CV: XjfuNWLQlEuxj6Mt.8
MS-ServerId: 030032362
Date: Tue, 13 Oct 2015 21:21:51 GMT

{
    "clientContext": {
        "client": "86b78998-d05a-487b-b380-6c738f6553ea"
    },
    "createdTime": "2015-10-13T21:21:51.1863494+00:00",
    "currencyCode": "USD",
```

```
    "isPIRequired": false,
    "language": "en-us",
    "market": "us",
    "orderId": "3eea1529-611e-4aee-915c-345494e4ee76",
    "orderLineItems": [{
        "availabilityId": "9RT7C09D5J3W",
        "beneficiary": {
            "identityType": "pub",
            "identityValue": "user1"
        },
        "billingState": "Charged",
        "currencyCode": "USD",
        "description": "Jewels, Jewels, Jewels - Consumable 2",
        "fulfillmentDate": "2015-10-13T21:21:51.639478+00:00",
        "fulfillmentState": "Fulfilled",
        "isPIRequired": false,
        "isTaxIncluded": true,
        "lineItemId": "2814d758-3ee3-46b3-9671-4fb3bdae9ffe",
        "listPrice": 0.0,
        "payments": [],
        "productId": "9NBLGGH5WVP6",
        "productType": "UnmanagedConsumable",
        "quantity": 1,
        "retailPrice": 0.0,
        "revenueRecognitionState": "None",
        "skuId": "0010",
        "taxAmount": 0.0,
        "taxType": "NoApplicableTaxes",
        "title": "Jewels, Jewels, Jewels - Consumable 2",
        "totalAmount": 0.0
    }],
    "orderState": "Purchased",
    "orderValidityEndTime": "2015-10-14T21:21:51.1863494+00:00",
    "orderValidityStartTime": "2015-10-13T21:21:51.1863494+00:00",
    "purchaser": {
        "identityType": "pub",
        "identityValue": "user1"
    },
    "testScenarios": "None",
    "totalAmount": 0.0,
    "totalTaxAmount": 0.0
}
```


# Error codes

Expand table

| Code | Error | Inner error code | Description |
|---|---|---|---|
| 401 | Unauthorized | AuthenticationTokenInvalid | The Azure AD access token is invalid. In some cases the details of the ServiceError |

| Code | Error | Inner error code | Description |
| --- | --- | --- | --- |
| | | | will contain more information, such as when the token is expired or the *appid* claim is missing. |
| 401 | Unauthorized | PartnerAadTicketRequired | An Azure AD access token was not passed to the service in the authorization header. |
| 401 | Unauthorized | InconsistentClientId | The *clientId* claim in the Microsoft Store ID key in the request body and the *appid* claim in the Azure AD access token in the authorization header do not match. |
| 400 | BadRequest | InvalidParameter | The details contain information regarding the request body and which fields have an invalid value. |

# Related topics

- Manage product entitlements from a service
- Query for products
- Report consumable products as fulfilled
- Renew a Microsoft Store ID key

# Feedback

# Get subscriptions for a user

Article • 10/20/2022

Use this method in the Microsoft Store purchase API to get the subscription add-ons that a given user has entitlements to use.

> **Note**
>
> This method can only be used by developer accounts that have been provisioned by Microsoft to be able to create subscription add-ons for Universal Windows Platform (UWP) apps. Subscription add-ons are currently not available to most developer accounts.

The Microsoft.StoreServices library provides the functionality of this method through the StoreServicesClient.RecurrenceQueryAsync API.

## Prerequisites

To use this method, you will need:

- An Azure AD access token that has the audience URI value `https://onestore.microsoft.com`.
- A Microsoft Store ID key that represents the identity of the user whose subscriptions you want to get.

For more information, see Manage product entitlements from a service.

## Request

### Request syntax

| Method | Request URI |
| --- | --- |
| POST | `https://purchase.mp.microsoft.com/v8.0/b2b/recurrences/query` |

### Request header

| Header | Type | Description |
| --- | --- | --- |

| Header | Type | Description |
| --- | --- | --- |
| Authorization | string | Required. The Azure AD access token in the form **Bearer** <*token*>. |
| Host | string | Must be set to the value **purchase.mp.microsoft.com**. |
| Content-Length | number | The length of the request body. |
| Content-Type | string | Specifies the request and response type. Currently, the only supported value is **application/json**. |

## Request body

| Parameter | Type | Description | Required |
| --- | --- | --- | --- |
| b2bKey | string | The Microsoft Store ID key that represents the identity of the user whose subscriptions you want to get. | Yes |
| continuationToken | string | If the user has entitlements to multiple subscriptions, the response body returns a continuation token when the page limit is reached. Provide that continuation token here in subsequent calls to retrieve remaining products. | No |
| pageSize | string | The maximum number of subscriptions to return in one response. The default is 25. | No |

## Request example

The following example demonstrates how to use this method to get the subscription add-ons that a given user has entitlements to use. Replace the *b2bKey* value with the Microsoft Store ID key that represents the identity of the user whose subscriptions you want to get.

JSON

```
POST https://purchase.mp.microsoft.com/v8.0/b2b/recurrences/query HTTP/1.1
Authorization: Bearer <your access token>
Content-Type: application/json
Host: purchase.mp.microsoft.com

{
  "b2bKey":  "eyJ0eXAiOiJ..."
}
```

# Response

This method returns a JSON response body that contains a collection of data objects that describe the subscription add-ons that the user has entitlements to use. The following example demonstrates the response body for a user who has an entitlement for one subscription.

JSON


```
{
  "items": [
    {
      "autoRenew":true,
      "beneficiary":"pub:gFVuEBiZHPXonkYvtdOi+tLE2h4g2Ss0ZId0RQOwzDg=",
      "expirationTime":"2017-06-11T03:07:49.2552941+00:00",
      "id":"mdr:0:bc0cb6960acd4515a0e1d638192d77b7:77d5ebee-0310-4d23-b204-
83e8613baaac",
      "lastModified":"2017-01-08T21:07:51.1459644+00:00",
      "market":"US",
      "productId":"9NBLGGH52Q8X",
      "skuId":"0024",
      "startTime":"2017-01-10T21:07:49.2552941+00:00",
      "recurrenceState":"Active"
    }
  ]
}
```


## Response body

The response body contains the following data.

| Value | Type | Description |
|---|---|---|
| items | array | An array of objects that contain data about each subscription add-on that the specified user has an entitlement to use. For more information about the data in each object, see the following table. |

Each object in the *items* array contains the following values.

| Value | Type | Description |
|---|---|---|
| autoRenew | Boolean | Indicates whether the subscription is configured to automatically renew at the end of the current subscription period. |
| beneficiary | string | The ID of the beneficiary of the entitlement that is associated with this subscription. |

| Value | Type | Description |
| --- | --- | --- |
| expirationTime | string | The date and time the subscription will expire, in ISO 8601 format. This field is only available when the subscription is in certain states. The expiration time usually indicates when the current state expires. For example, for an active subscription, the expiration date indicates when the next automatic renewal will occur. |
| expirationTimeWithGrace | string | The date and time the subscription will expire including the grace period, in ISO 8601 format. This value indicates when the user will lose access to the subscription after the subscription has failed to automatically renew. |
| id | string | The ID of the subscription. Use this value to indicate which subscription you want to modify when you call the change the billing state of a subscription for a user method. |
| isTrial | Boolean | Indicates whether the subscription is a trial. |
| lastModified | string | The date and time the subscription was last modified, in ISO 8601 format. |
| market | string | The country code (in two-letter ISO 3166-1 alpha-2 format) in which the user acquired the subscription. |
| productId | string | The Store ID for the product that represents the subscription add-on in the Microsoft Store catalog. An example Store ID for a product is 9NBLGGH42CFD. |
| skuId | string | The Store ID for the SKU that represents the subscription add-on the Microsoft Store catalog. An example Store ID for a SKU is 0010. |
| startTime | string | The start date and time for the subscription, in ISO 8601 format. |

| Value | Type | Description |
| --- | --- | --- |
| recurrenceState | string | One of the following values:<br>• **None**: This indicates a perpetual subscription.<br>• **Active**: The subscription is active and the user is entitled to use the services.<br>• **Inactive**: The subscription is past the expiration date, and the user turned off the automatic renew option for the subscription.<br>• **Canceled**: The subscription has been purposefully terminated before the expiration date, with or without a refund.<br>• **InDunning**: The subscription is in *dunning* (that is, the subscription is nearing expiration, and Microsoft is trying to acquire funds to automatically renew the subscription).<br>• **Failed**: The dunning period is over and the subscription failed to renew after several attempts.<br><br>**Note:**<br><br>• **Inactive/Canceled/Failed** are terminal states. When a subscription enters one of these states, the user must repurchase the subscription to activate it again. The user is not entitled to use the services in these states.<br>• When a subscription is **Canceled**, the expirationTime will be updated with the date and time of the cancellation.<br>• The ID of the subscription will remain the same during its entire lifetime. It will not change if the auto-renew option is turned on or off. If a user repurchases a subscription after reaching a terminal state, a new subscription ID will be created.<br>• The ID of a subscription should be used to execute any operation on an individual subscription.<br>• When a user repurchases a subscription after cancelling or discontinuing it, if you query the results for the user you will get two entries: one with the old subscription ID in a terminal state, and one with the new subscription ID in an active state.<br>• It's always a good practice to check both recurrenceState and expirationTime, since updates to recurrenceState can potentially be delayed by a few minutes (or occasionally hours). |
| cancellationDate | string | The date and time the user's subscription was cancelled, in ISO 8601 format. |

# Related topics

- Manage product entitlements from a service
- Change the billing state of a subscription for a user
- Query for products
- Report consumable products as fulfilled
- Renew a Microsoft Store ID key
- Microsoft.StoreServices library (GitHub)